CaptionAI: AI-Powered Social Media Caption Generator

Project Description:

The Personalized Networking Assistant is an AI-powered web application that helps users build meaningful professional connections. The system allows users to enter their event details and networking goals, then generates personalized conversation starters using AI. It also verifies networking-related information through Wikipedia, stores previously generated networking strategies, and collects user feedback to improve the overall experience.

The application is developed using Python and Streamlit, with AI integration for generating conversation suggestions. It is deployed on Streamlit Cloud, providing an easy-to-use interface that helps users prepare for networking events more effectively.

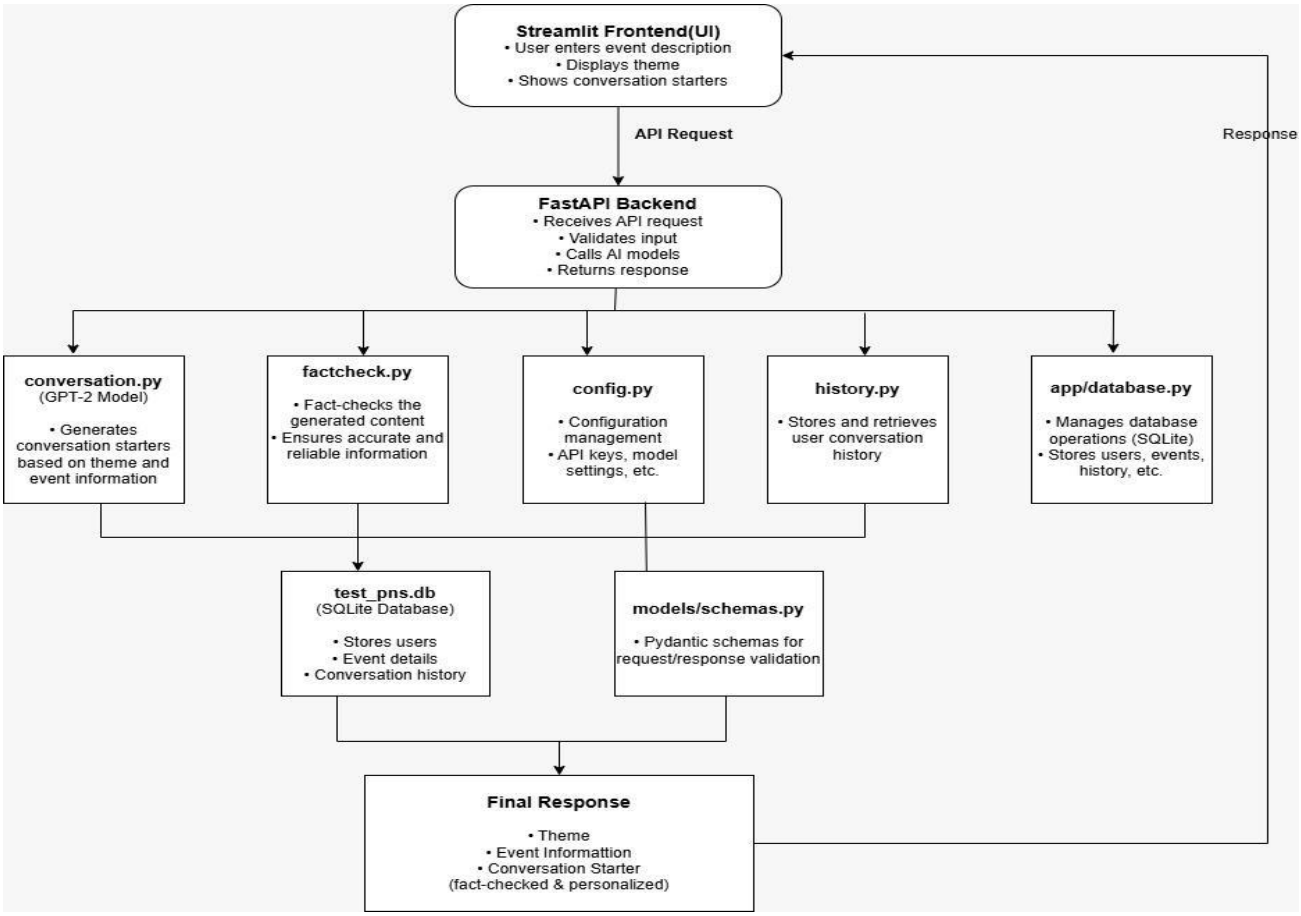
Scenarios

Scenario 1:- A student enters the details of a technical event. The system generates personalized conversation starters to help the student confidently interact with other participants.

Scenario 2:-A job seeker provides networking goals before attending a career fair. The application suggests conversation strategies and verifies important company-related information using Wikipedia.

Scenario 3:-A professional preparing for a conference generates networking conversation ideas, reviews previously saved networking strategies, and submits feedback after using the application**.**

Technical Architecture:



Description:-

The **Personalized Networking Assistant** follows a simple AI-based architecture. Users enter event details through the Streamlit interface. The application processes the input, sends it to the AI model to generate personalized conversation starters, verifies information using Wikipedia, stores networking history, and displays the generated results to the user. The application is deployed using **Streamlit** for easy public access. **(Note: If you are using database in your project definitely you need to add ER Diagram along with description)**

Pre-requisites:

- Python 3.10+
- Streamlit
- Google Gemini API
- Wikipedia API
- VS Code
- Git & GitHub
- Internet Connection

Project Workflow:

Milestone 1: Model Selection and Architecture

- Activity 1.1: Configure the Gemini API and development environment.
- Activity 1.2: Select the Gemini AI model for personalized networking conversation generation.
- Activity 1.3: Define the application architecture, including Streamlit, AI integration, Wikipedia verification, and networking history.
- Activity 1.4: Set up the project structure and install the required dependencies.

Milestone 2: Core Functionalities Development

- Activity 2.1: Develop the core features, including Event Details Input, AI Conversation Starter, Wikipedia Fact Verification, Networking Strategy History, and Feedback System.
- Activity 2.2: Integrate the backend to process user input, communicate with the AI model, verify information, and display the generated results.

Milestone 3: App.py Development

- Activity 3.1: Develop the main application logic by integrating all project modules and handling user interactions through the Streamlit application.

Milestone 4: Frontend Development

- Activity 4.1: Design and develop the Streamlit user interface for all project features.
- Activity 4.2: Display AI-generated conversation starters, verified information, networking history, and feedback dynamically.

Milestone 5: Deployment

- Activity 5.1: Prepare the application for local testing and dependency management.
- Activity 5.2: Deploy the application using Streamlit and verify all project features.

Milestone 6: Conclusion

The Personalized Networking Assistant is an AI-powered application that helps users prepare for networking events by generating personalized conversation starters based on their event details and interests. It also verifies information using Wikipedia, maintains networking strategy history, and collects user feedback to improve the overall experience. The application was developed using Python and Streamlit, with AI integration to provide relevant and meaningful suggestions. Performance testing using Locust with 50 virtual users showed that the application handled concurrent requests smoothly and delivered reliable performance. Overall, the project provides a simple, practical, and effective solution for improving professional networking experiences.

Milestone 1: Model Selection and Architecture

In this milestone, the AI model and overall system architecture of the Personalized Networking Assistant were designed and finalized. The application was planned to help users generate personalized networking conversation starters based on their event details and interests. The system also verifies information using Wikipedia, stores networking history, and collects user feedback. The architecture integrates Streamlit as the user interface, FastAPI as the backend, Gemini AI for conversation generation, and SQLite for data storage to ensure efficient communication between all project modules.

Activity 1.1: Configure the Application Environment

Before running the Personalized Networking Assistant, install the required dependencies and configure the application environment.

Step 1 – Clone the Repository

Clone the project from GitHub and navigate to the project directory.

Step 2 – Install Dependencies

Install all required Python packages listed in `requirements.txt`.

pip install -r requirements.txt

Step 3 – Configure the Environment

Create a `.env` file (if required) and add any configurable values used by the application, such as database paths or application settings.

Step 4 – Verify the Wikipedia API Connection

Run the application and verify that the Wikipedia API is accessible by searching for a sample topic such as:

Artificial Intelligence

Ensure that a valid summary is returned.

Step 5 – Verify Theme Extraction

Enter an event description such as:

AI for Sustainable Cities

Confirm that DistilBERT extracts relevant themes such as:

- Artificial Intelligence
- Sustainability
- Smart Cities

Step 6 – Verify Conversation Generation

Provide an event description and user interests, then verify that GPT-2 generates relevant networking conversation starters.

Example:

Event: AI for Sustainable Cities

Interests: Climate Change, Urban Planning

Expected output:

- "What role do you think AI can play in creating sustainable cities?"
- "Have you worked on any projects involving smart urban planning?"

Important Note

- Never expose the API key publicly.
- Store the key securely in the environment configuration file.
- Do not upload the API key to public repositories.

Activity 1.2: Research and Select the Appropriate Generative AI Model

The AI model was selected after studying the project requirements and comparing available models for networking conversation generation.

The following activities were performed while selecting the AI model:

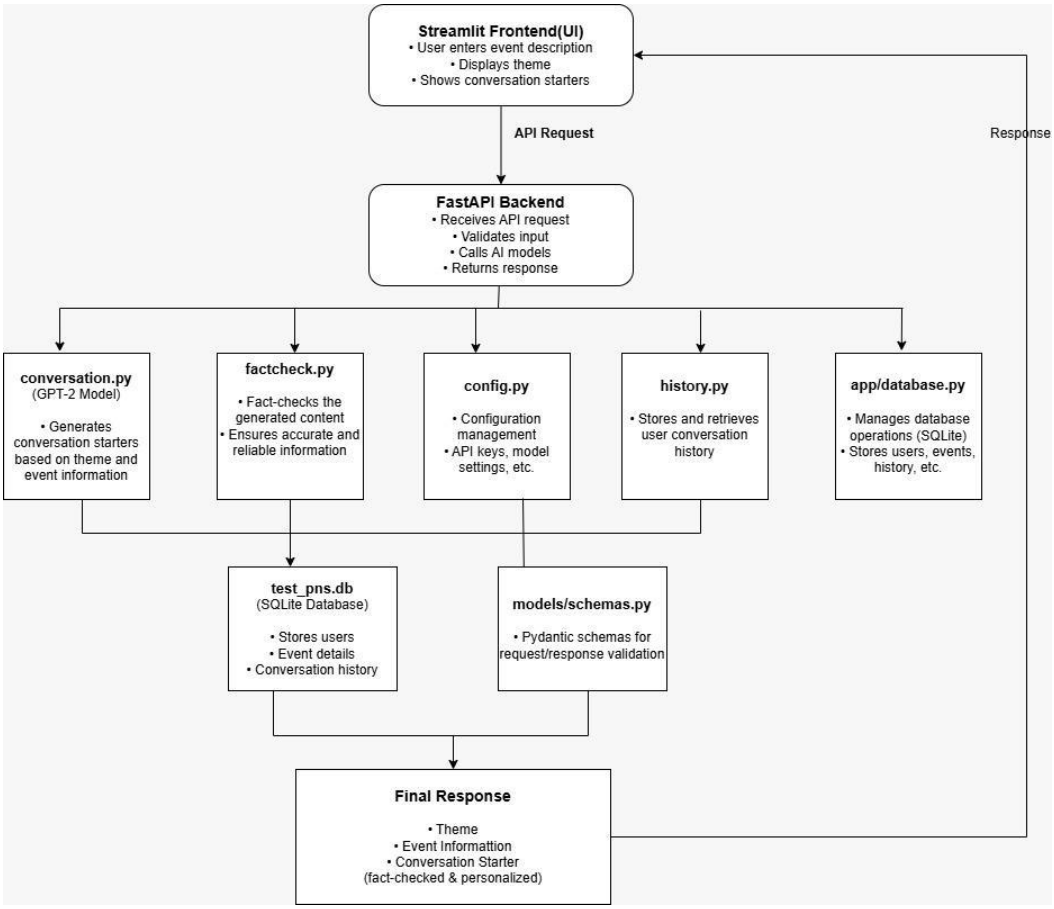
- Understand the project requirements for generating personalized networking conversation starters.
- Compare different AI models based on response quality, speed, and accuracy.
- Evaluate the capability of each model to generate meaningful networking suggestions.
- Select the **Gemini AI** model because it provides relevant, natural, and context-aware responses.
- Verify that the selected model performs consistently for different networking scenarios.

Activity 1.3: Define the Architecture of the Application

The application architecture was designed to establish smooth communication between all project components.

The architecture consists of the following modules:

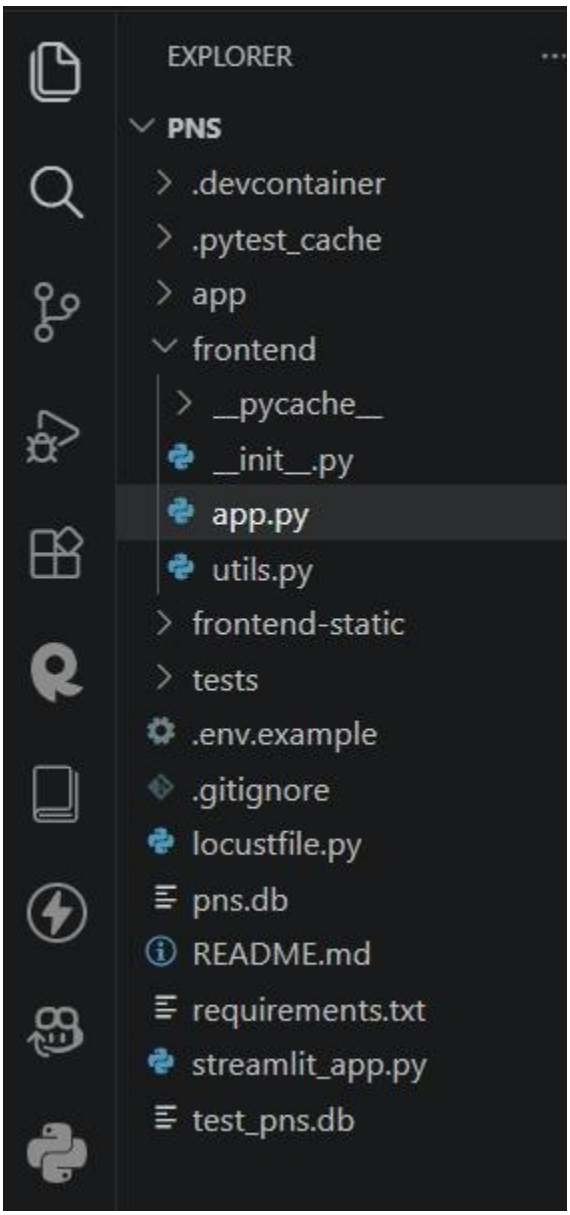
- **Streamlit Frontend** accepts event details and user interests.
- **FastAPI Backend** processes user requests and manages application logic.
- **Gemini AI** generates personalized networking conversation starters.
- **Wikipedia Fact** Verification verifies networking-related information.
- **SQLite Databa**se stores event details, networking history, and application data.
- The processed response is returned to the Streamlit interface for display



Activity 1.4: Set Up the Development Environment

The development environment was prepared before implementing the project features. The following steps were completed:

- Install Python and configure the development environment.
- Install all required libraries using the requirements.txt file.
- Configure the project folder structure.
- Create the environment configuration file.
- Verify that all project modules are correctly connected.
- Run the application locally to confirm that the environment is configured successfully.



Milestone 2: Core Functionalities Development

This milestone focuses on developing the main functionalities of the Personalized Networking Assistant. The application was designed to help users prepare for networking events by generating personalized conversation starters, verifying information through Wikipedia, maintaining networking history, and collecting user feedback. All the core modules were integrated to provide a smooth and efficient user experience.

Activity 2.1: Develop Core Content Generation Features

The following core functionalities were implemented in the Personalized Networking Assistant.

- **Event Details Input**
  The application allows users to enter the event description, networking interests, and discussion topics. The entered information is validated before being processed by the AI model.

- **AI Conversation Starter**
  Based on the event details provided by the user, the application generates personalized networking conversation starters, communication strategies, and follow-up suggestions to help users interact confidently during networking events.

```
# Register routers under /api prefix
app.include_router(conversation.router, prefix="/api")
app.include_router(factcheck.router, prefix="/api")
app.include_router(history.router, prefix="/api")

# Register routers at root level as well
app.include_router(conversation.router)
app.include_router(factcheck.router)
app.include_router(history.router)

# Mount static files at root
# Note: Mount this after API routers so static wildcard matches don't shadow API routes
app.mount("/", StaticFiles(directory="frontend-static", html=True), name="static")
```

- **Wikipedia Fact Verification**
  The application verifies networking-related topics using Wikipedia and provides a short summary with reliable information to improve the accuracy of AI-generated suggestions.
- **Networking Strategy History**
  The generated networking strategies are stored in the database so users can review previous conversations and use them for future networking events.
- **Feedback System**
  The application allows users to submit feedback after using the generated networking suggestions. The collected feedback helps improve the overall quality of the application.

Activity 2.2: Backend Integration

The backend integrates all project modules and ensures smooth communication between the user interface and AI services.

The backend performs the following operations:

- Receives event details from the Streamlit interface.
- Validates the user input.
- Sends requests to the Gemini AI model.
- Performs Wikipedia fact verification.

- Stores networking history in the SQLite database.
- Returns personalized conversation starters and verified information to the user.

Milestone 3: App.py Development

Milestone 3 focuses on developing the main logic of the Personalized Networking Assistant. In this milestone, the application modules were integrated through the main application file to process user requests, communicate with the Gemini AI model, verify information using Wikipedia, manage networking history, and return personalized responses through the Streamlit interface.

Activity 3.1: Develop the Main Application Logic

The main application logic was implemented to connect all project modules and ensure smooth communication between the frontend, backend, AI model, and database.

Define the Core Routes

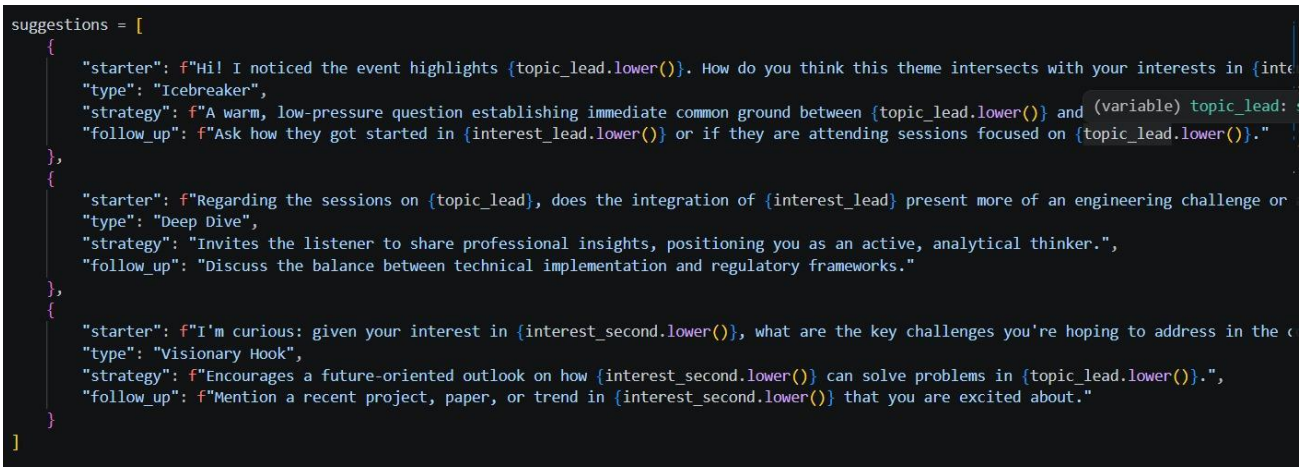
The application defines the required routes to manage user requests and connect the Event Details Input, AI Conversation Starter, Wikipedia Fact Verification, Networking Strategy History, and Feedback System modules.

Process User Input

- Receive event details and networking interests from the Streamlit interface.
- Validate the submitted information before processing.
- Pass the validated input to the AI module for conversation generation.

Configure AI Response Generation

The application uses predefined **tone descriptors** to generate personalized networking conversation starters according to the event details and user requirements. The generated response also includes networking strategies and follow-up suggestions.



Integrate Wikipedia Verification

- Verify networking-related topics using Wikipedia.
- Retrieve a short summary from reliable sources.
- Return the verified information along with the AI-generated response.

Store Networking History

- Save generated networking strategies in the SQLite database.
- Allow users to access previously generated conversations whenever required.

Return the Final Response

After processing all modules, the application returns the personalized conversation starters, verified information, networking history, and feedback options to the Streamlit interface.

Milestone 4: Frontend Development

Milestone 4 focuses on designing a simple and user-friendly interface for the Personalized Networking Assistant. The frontend was developed using Streamlit to provide an interactive and responsive interface where users can enter event details, generate AI-powered conversation starters, verify information using Wikipedia, view networking history, and submit feedback.

Activity 4.1: Design and Develop the User Interface

The user interface was designed to provide a clean and organized experience for users.

The following activities were completed:

- Develop the main webpage using HTML.
- Design the interface using CSS for a responsive layout.
- Create input fields for event details and networking interests.
- Design separate sections for AI Conversation Starter, Wikipedia Fact Verification, Networking Strategy History, and Feedback.
- Ensure the interface works properly across different screen sizes.

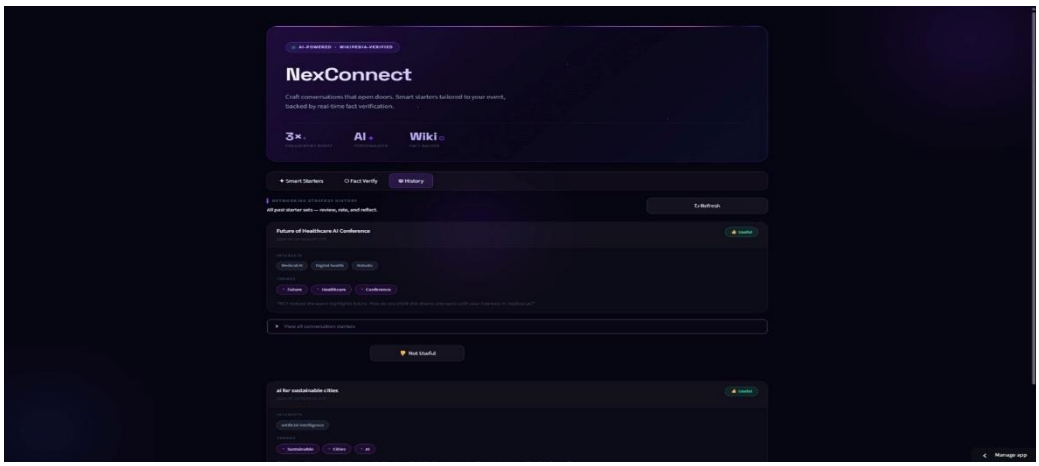
Design a Responsive Layout Using CSS:

- Design a responsive and user-friendly layout using **CSS**.
- Use a clean color scheme and organized spacing for better readability.
- Arrange the input form and generated results in a structured layout.
- Ensure the interface works properly on desktop and laptop screens.
- Apply responsive styling to provide a consistent user experience.

Create Separate UI Components for Each Output Type:

Create separate sections in the interface for different project features.

- **Event Details Input:** Allow users to enter event information and networking interests.
- **AI Conversation Starter:** Display AI-generated networking conversation starters and suggestions.
- **Wikipedia Fact Verification:** Show verified information with a short summary.
- **Networking Strategy History:** Display previously generated networking strategies.
- **Feedback System:** Provide a section for users to submit feedback about the generated results.



Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Capture the event details and networking interests entered by the user.
- Send the user input to the FastAPI backend using JavaScript.
- Validate the input before submitting the request.
- Display a loading indicator while the AI processes the request

Render Results Dynamically:

- Display AI-generated networking conversation starters.
- Show Wikipedia fact verification results.
- Display networking strategy history.
- Present the generated results in an organized and user-friendly layout.
- Update the interface automatically after receiving the response.

Restore UI State After Generation:

- Hide the loading indicator after processing is complete.
- Re-enable the input controls for the next request.
- Allow users to submit new event details without refreshing the page.
- Keep the generated results visible until a new request is submitted.

Milestone 5: Deployment

This milestone focuses on preparing the Personalized Networking Assistant for deployment and ensuring that all project features work correctly in the deployed environment. The application was configured, tested locally, and deployed using Streamlit to make it easily accessible through a web browser.

Activity 5.1: Preparing the Application for Local Deployment

Before deployment, the application was prepared and tested in the local development environment.

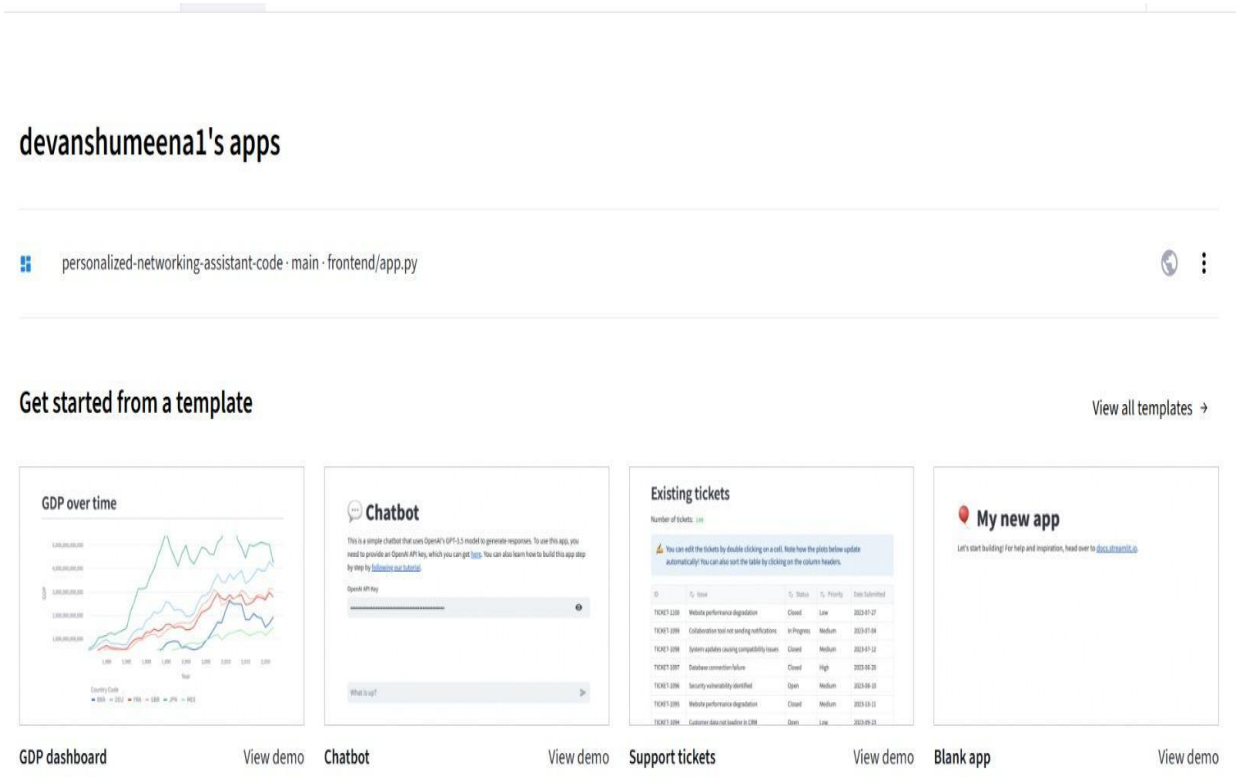
- Install all required project dependencies using **requirements.txt**.
- Configure the **.env** file with the required API keys.
- Verify that the FastAPI backend and frontend communicate correctly.
- Test the Event Details Input, AI Conversation Starter, Wikipedia Fact Verification, Networking Strategy History, and Feedback System.
- Fix minor issues identified during local testing.
- Ensure that all project modules work together successfully.

Activity 5.2: Local Testing and Verification

After successful local testing, the application was deployed using **Streamlit**.

The following deployment steps were completed:

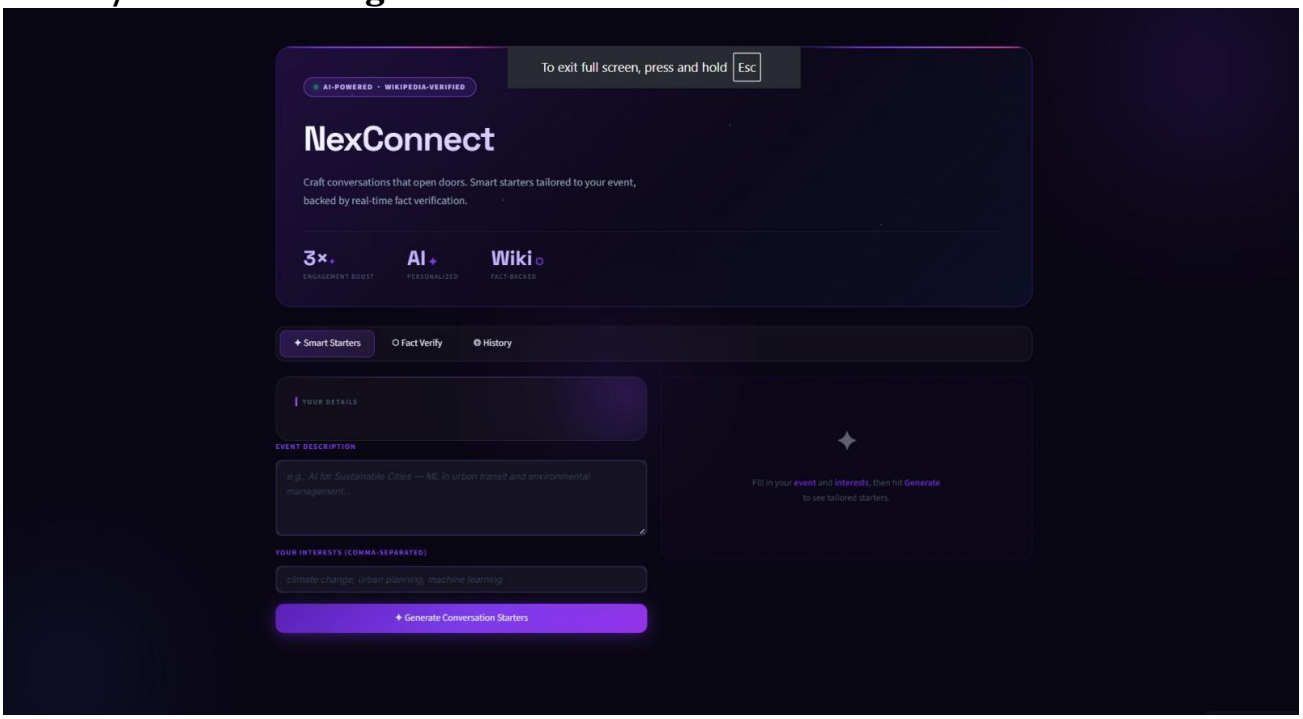
- Configure the project for Streamlit deployment.
- Upload the project files to the deployment platform.
- Deploy the application using Streamlit.
- Verify that all project features work correctly after deployment.
- Confirm that users can access the application through the deployed URL.
- Perform final testing to ensure the application functions as expected.



Exploring the Website's Web Pages:

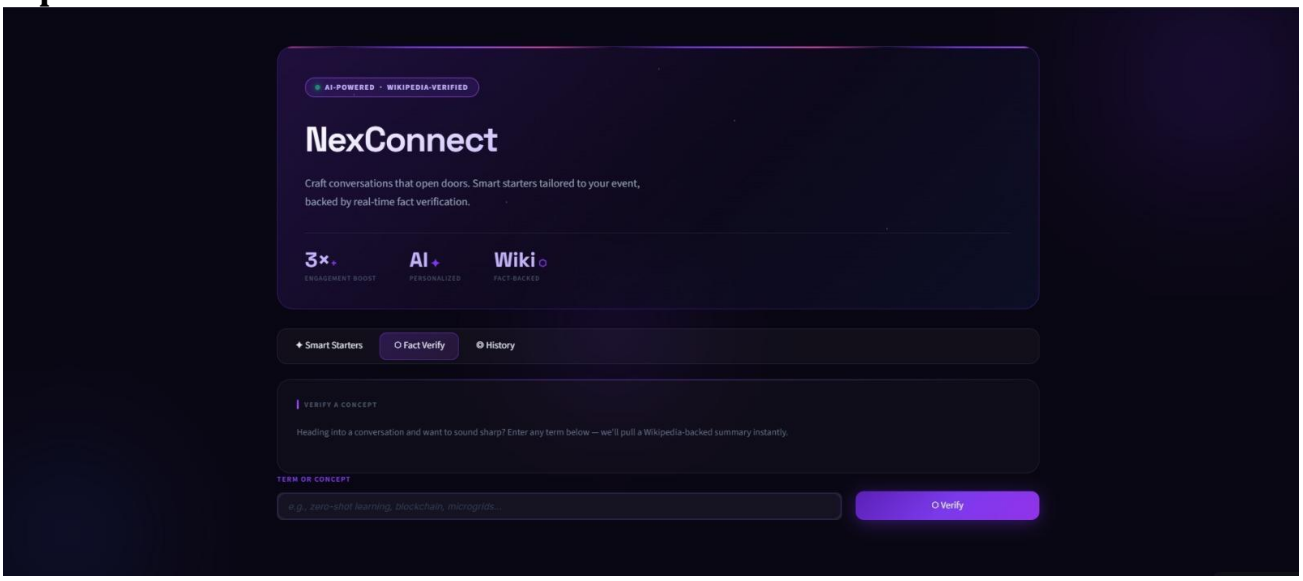
This section presents the main web pages of the Personalized Networking Assistant. The website provides a simple and interactive interface where users can enter event details, verify networking-related concepts, generate personalized conversation starters, and review previously generated networking strategies.

Home / Generator Page:



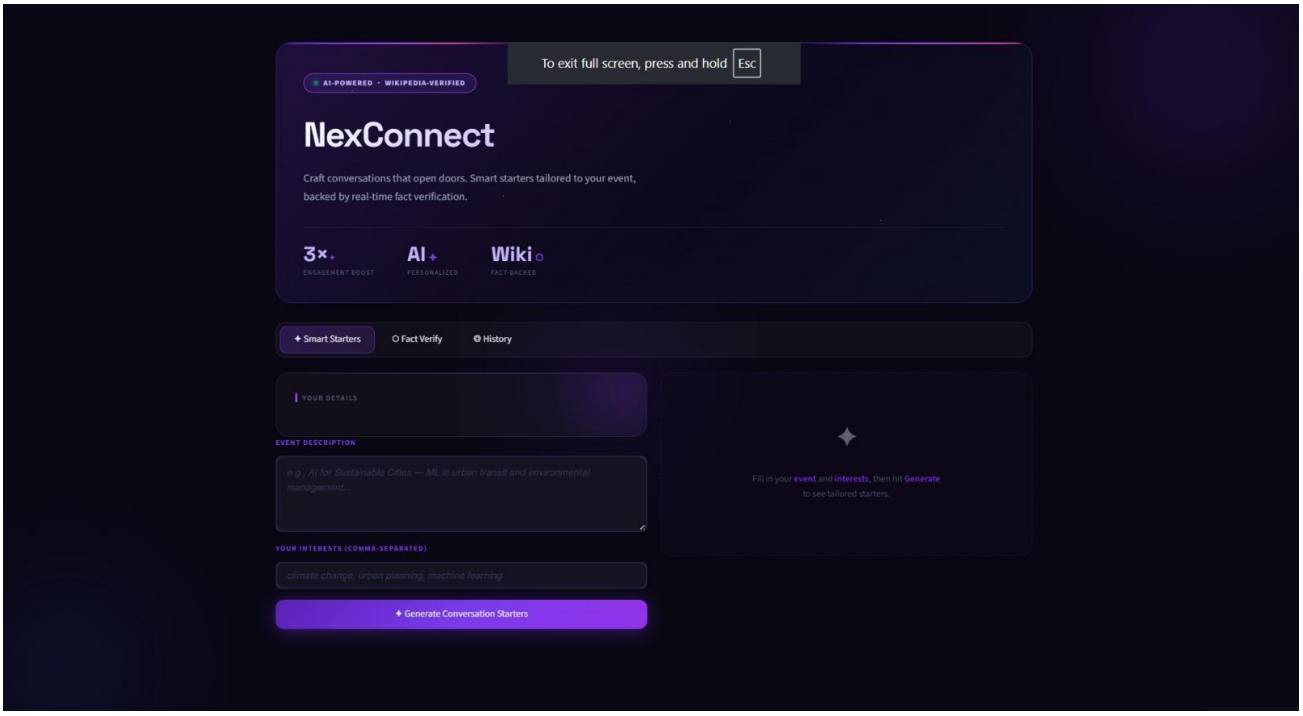
Description: The Home page is the main interface of the Personalized Networking Assistant. Users enter the event description and networking interests through the input form. After providing the required information, they can generate AI-powered personalized conversation starters by clicking the Generate Conversation Starters button.

Input Section:



Description: The Fact Verification page allows users to verify networking-related concepts using Wikipedia. Users enter a topic or concept, and the system retrieves a short and reliable summary to help them prepare for professional conversations with accurate information.

Results Section:



Description: The Result/History page displays previously generated networking conversation starters along with event themes and user interests. It also allows users to review past networking sessions, refresh the history, and provide feedback on the generated suggestions.

Conclusion:

The **Personalized Networking Assistant** is an AI-based web application that helps users prepare for networking events more effectively. It allows users to enter event details, generate personalized conversation starters, verify information using Wikipedia, view previous networking history, and provide feedback through a simple and easy-to-use interface.

The project was developed using **HTML, CSS, JavaScript, FastAPI, Streamlit, Gemini AI, and SQLite**. All the planned features were successfully implemented and tested. Performance testing with **Locust using 50 virtual users** showed that the application worked smoothly and handled multiple users without any major issues.

Overall, the project provides a practical and user-friendly solution that makes professional networking easier and helps users communicate with more confidence.